A Project Report on

"Real-Time Vehicles Tracking System for Urban Public Transport"

Submitted in the partial fulfillment of the requirements for the award of the Degree of

Bachelor of Computer Application

Semester - VI (2025-26)

Submitted by

230801640 – Umaraniya Jeet Hareshbhai
230801604 – Sureliya Darshan Divyangbhai

Guided by

Mr. Mehul Makwana

Submitted to

Faculty of Science

Department of Computer Science

Atmiya University

Replace the sample certificate with your original certificate.

Project ID: 25BCA5C10-95 **Date:** 29-11-2025

CERTIFICATE

This is to certify that,

Mr. / Ms. / Mrs. Darshan Ghetiya of

Bachelor of Computer Application

Semester V

Has satisfactorily completed the project on

Automated Library Management and Information System

For, Department of Computer Science,

Faculty of Science, ATMIYA University, Rajkot, Gujarat (India).

Signature
(Project Guide)

Signature
(Head of the Department)

DECLARATION

I hereby declare that the project report titled "**Real-Time Vehicles Tracking System for Urban Public Transport**" is my original work and has been completed under the guidance of **Mr. Mehul Makwana**. This project is submitted in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Application, Semester – VI (2025–26) under the Faculty of Science, Department of Computer Science, Atmiya University, Rajkot, Gujarat-India.

I also declare that this work has not been submitted to any other university or institution for any degree, diploma, or certificate.

Name of Student: Umaraniya Jeet Hareshbhai

Enrollment No.: 230801640

Signature:

Date:

ACKNOWLEDGEMENT

I express my deep sense of gratitude to the following respected authorities of Atmiya University for their constant support, guidance, and encouragement throughout the completion of this project.

I am sincerely thankful to **P. P. Tyagvallabh Swami** (President, Atmiya University) for providing an excellent educational environment and continuous encouragement toward academic excellence.

I extend my gratitude to **Dr. D. D. Vyas** (Registrar, Atmiya University) for ensuring smooth academic processes and administrative support that contributed to the successful completion of this project.

I am grateful to **Er. R. S. Tank** (Dean, Faculty of Science) for his valuable support, motivation, and guidance throughout the duration of the program.

My heartfelt thanks to **Dr. D. P. Gohel** (Head, Department of Computer Science) for providing necessary facilities, timely support, and continuous encouragement during the project work.

I would like to express my special appreciation and sincere thanks to my project guide **Mr. Mehul Makwana**, whose expert guidance, patience, and valuable suggestions helped me complete this project successfully.

I am also thankful to all the faculty members of the Department of Computer Science for their support, feedback, and academic insights throughout the course.

Finally, I gratefully acknowledge the constant support, motivation, and encouragement of my family and friends, without which this project would not have been completed.

ABSTRACT

In the rapidly evolving logistics and transportation industry, the ability to monitor fleet movements in real-time is no longer a luxury but a necessity for operational efficiency and security. Traditional methods of vehicle tracking, often reliant on manual check-ins or delayed communication, fail to provide the granular visibility required for modern supply chain management. This project, the **Vehicle Tracking System**, addresses these challenges by delivering a web-based solution for real-time fleet monitoring and management.

The system is architected using the MERN stack (MongoDB, Express.js, React.js, Node.js), leveraging the power of JavaScript across the full development stack. A critical component of the architecture is the integration of **Socket.IO**, which facilitates bidirectional, low-latency communication between the server and clients, enabling live location updates without the need for constant page refreshes. Furthermore, the frontend utilizes **Leaflet.js** to render interactive, open-source maps that visualize vehicle positions and routes dynamically.

Key functionalities include role-based access control for Administrators and Drivers, CRUD operations for vehicle and route management, and a historical data playback feature. By providing a centralized dashboard for tracking, this project aims to reduce operational costs, improve delivery time estimation, and enhance the overall safety of fleet operations. The outcome is a scalable, responsive, and user-friendly application that demonstrates the practical application of modern web technologies in solving real-world logistical problems.

TABLE OF CONTENT

SN	Chapter	Page No.
	Certificate	I
	Declaration	II
	Acknowledgment	III
	Abstract	IV
	Table of Contents	V
1	Introduction	1
2	Problem Definition	3
3	Objectives of the Study	4
4	System Analysis	5
5	System Design	
	5.1 Data Flow Diagrams (DFD)	8
	5.2 Use Case Diagrams	9
	5.3 Class Diagrams	10
	5.4 Activity Diagrams	11
	5.5 System Architecture Diagrams	12
	5.6 ER Diagrams	13
	5.5 UML Sequence Diagrams	14
6	Implementation	15
7	Testing	17
8	Output Screenshots	19
9	Conclusion	27
10	Future Scope	28
11	Bibliography / References	29
12	Appendix	30

INTODUCTION

The logistics and transportation sector serves as the backbone of the global economy, ensuring the timely movement of goods and services across vast distances. As managing fleets becomes increasingly complex, the demand for sophisticated tracking solutions has surged. The **Vehicle Tracking System** proposed in this project is a web-based application designed to provide comprehensive tools for monitoring vehicle locations, managing routes, and overseeing driver activities in real-time.

Modern web technologies have evolved to support complex, real-time data processing, making it possible to build tracking systems that are both accessible via standard web browsers and highly responsive. This project utilizes the MERN stack—comprising MongoDB, Express.js, React.js, and Node.js—to create a unified JavaScript development environment. This choice allows for seamless data flow (JSON) between the client and server, efficient handling of concurrent connections, and a reactive user interface that updates instantaneously as new data arrives.

- **PURPOSE AND SCOPE**

The primary purpose of this project is to develop a cost-effective and scalable Vehicle Tracking System that eliminates the blindness often associated with fleet management. By visualizing vehicle locations on a map in real-time, administrators can make informed decisions regarding route optimization, emergency response, and scheduling.

The scope of the project encompasses:

- **REAL-TIME MONITORING:** Tracking the precise location of vehicles using simulated GPS coordinates.
- **FLEET MANAGEMENT:** A centralized interface for adding, updating, and removing vehicles and drivers.
- **ROUTE PLANNING:** Defining start and end points for journeys and monitoring progress.
- **HISTORICAL ANALYSIS:** Storing location data to allow for retrospective analysis of routes taken.

- **USER MANAGEMENT:** Secure authentication and authorization systems distinguishing between Admin and Driver roles.

BACKGROUND INFORMATION

Historically, vehicle tracking was a capital-intensive implementation reserved for large logistics corporations, often requiring expensive proprietary hardware and satellite subscriptions. With the advent of widespread cellular networks and GPS technology, combined with the democratization of web development tools, creating custom tracking solutions has become feasible for smaller enterprises and educational projects. This system builds upon these advancements, abstracting the hardware layer to focus on the software architecture required to handle high-frequency location data streams and present them in a user-friendly format.

PROBLEM DEFINITION

In current non-digitized or semi-digitized transport operations, fleet managers face a significant "black box" regarding their assets once they leave the depot. The lack of visibility leads to several critical operational inefficiencies.

Firstly, **communication delays** are rampant. Managers currently rely on phone calls or text messages to ascertain a driver's location. This manual process is distracting for drivers, dangerous while driving, and often results in inaccurate or outdated information by the time it reaches the decision-maker.

Secondly, **inefficient routing** is a major cost driver. Without real-time data, it is impossible to dynamic re-route vehicles to avoid traffic congestion or to assign the nearest available vehicle to a new pickup request. This results in higher fuel consumption, increased wear and tear on vehicles, and longer delivery times.

Thirdly, **security and compliance** risks are elevated. In the event of vehicle theft or unauthorized use, the absence of a tracking system means recovery efforts are severely hampered. Furthermore, verifying that drivers are adhering to assigned routes and schedules is nearly impossible without continuous monitoring.

Therefore, there is a pressing need for a technical solution that automates the tracking process. A centralized digital system that aggregates location data and presents it visually can solve these problems by providing a "single pane of glass" view of the entire fleet operation, reducing manual intervention and increasing overall transparency.

OBJECTIVES OF THE STUDY

The primary objective of this project is to design and implement a robust web-based Vehicle Tracking System. The specific objectives are as follows:

- **TO IMPLEMENT REAL-TIME LOCATION TRACKING:** To utilize Socket.IO and Leaflet maps to display live vehicle movements on an interactive interface without requiring manual page reloads.
- **TO DEVELOP A ROLE-BASED ACCESS CONTROL SYSTEM:** To create a secure authentication mechanism that differentiates between Administrators (who manage the fleet) and Drivers (who broadcast location data).
- **TO ENABLE COMPREHENSIVE FLEET MANAGEMENT:** To provide CRUD (Create, Read, Update, Delete) capabilities for managing vehicle profiles, including registration numbers, types, and current status.
- **TO FACILITATE ROUTE MANAGEMENT:** To allow administrators to define and assign specific routes to vehicles to ensure adherence to operational plans.
- **TO MAINTAIN HISTORICAL USAGE DATA:** To design a database schema capable of storing timeseries location data for post-trip analysis and reporting.
- **TO ENSURE SYSTEM RESPONSIVENESS:** To build a responsive frontend using React.js that functions seamlessly across different device sizes, including desktops and tablets.

SYSTEM ANALYSIS

- **EXISTING SYSTEM**

The existing system in many small-to-medium logistics setups relies heavily on manual processes.

- **WORKING METHOD:** Dispatchers assign trips on paper or spreadsheets. Drivers report their status via phone calls at varying intervals.
- **LIMITATIONS:** Information is gathered asynchronously and is often outdated. There is no visual representation of the fleet's distribution.
- **RISKS AND INEFFICIENCIES:** High dependency on human honesty and punctuality. Zero ability to recover stolen assets or prove service delivery times objectively. Data entry errors in spreadsheets lead to poor record-keeping.
- **PROPOSED SYSTEM**

The proposed automated Vehicle Tracking System digitizes the entire workflow.

- **SYSTEM WORKFLOW:** Vehicles (simulated or connected via mobile web) transmit their GPS coordinates to a central Node.js server. The server processes this data and broadcasts it via Web Sockets to the Admin dashboard.
- **ADVANTAGES:**
- **IMMEDIACY:** Updates occur in milliseconds.
- **ACCURACY:** Coordinates are plotted directly on a map, removing ambiguity.
- **AUTOMATION:** History is logged automatically without driver input.
- **SOLUTION STRATEGY:** By shifting from manual reporting to automated polling, the system removes the human error factor and drastically reduces the communication overhead between drivers and managers.

- **FEASIBILITY STUDY**
- **TECHNICAL FEASIBILITY**

The project uses the MERN stack, which is open-source, widely supported, and robust. Node.js is particularly well-suited for I/O-heavy applications like real-time tracking due to its non-blocking event-driven architecture. Socket.IO provides a reliable layer over WebSocket's. The technical requirements are well within the capabilities of standard modern web development practices.

- **ECONOMIC FEASIBILITY**

The system is economically viable as it relies on open-source technologies (MongoDB, React, Leaflet). There are no licensing fees for the core software stack. The cost of implementation is primarily time and development effort. Deploying such a system can lead to significant cost savings in fuel and time for the end-user, ensuring a high return on investment.

- **OPERATIONAL FEASIBILITY**

The system is designed with a user-friendly interface (UI). Since it is a web application, it requires no complex installation on client machines—only a web browser is needed. Deployment is straightforward on cloud platforms. The learning curve for end-users (admins and drivers) is minimal due to the intuitive design of the dashboard.

- **REQUIREMENT SPECIFICATION**
  - **HARDWARE REQUIREMENTS**

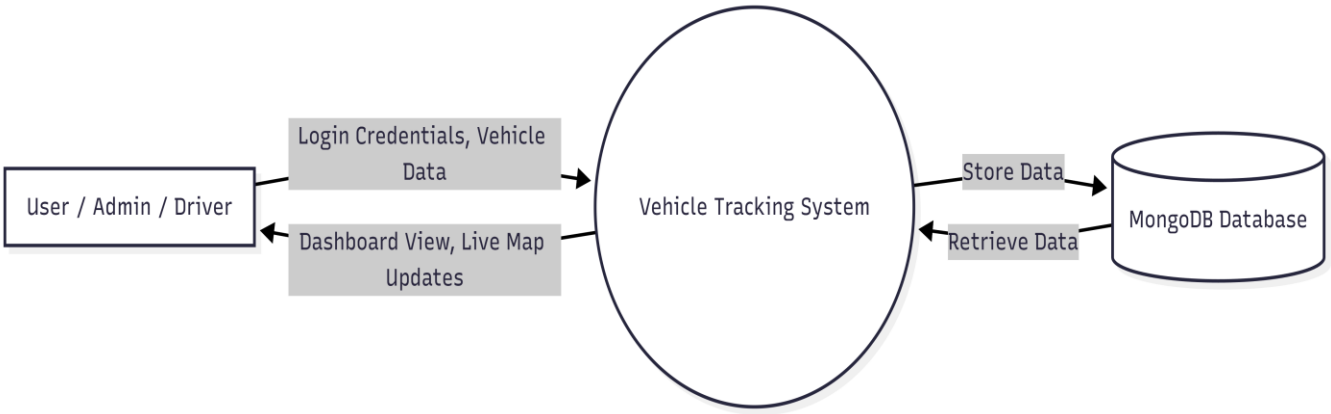
Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 or equivalent	Intel Core i5 or higher
RAM	4 GB	8 GB or higher
Storage	256 GB HDD/SSD	512 GB SSD
Internet	Stable Broadband Connection	High-speed Fiber/4G

  - **SOFTWARE REQUIREMENTS**

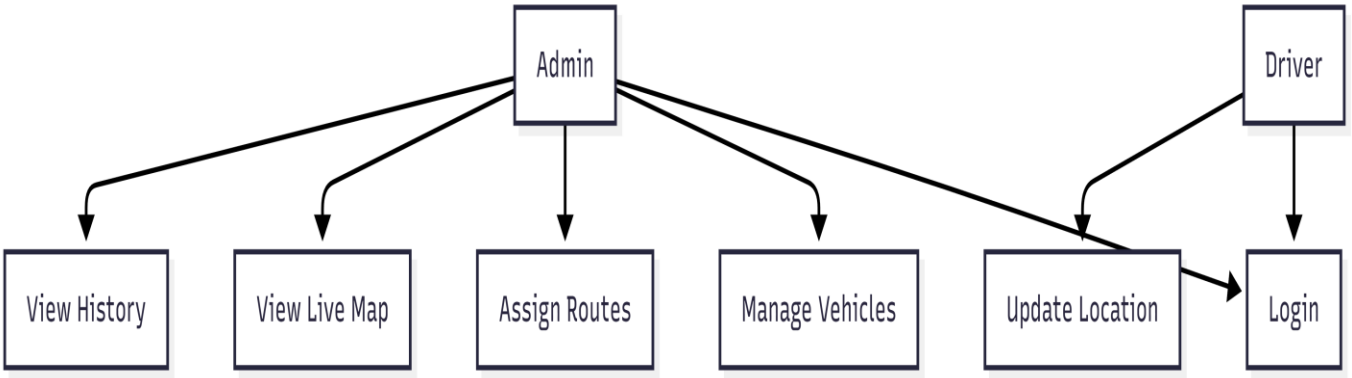
Component	Specification
Operating System	Windows 10/11, Linux (Ubuntu), or macOS
Language	JavaScript (ES6+)
Runtime Environment	Node.js (v14.x or higher)
Database	MongoDB (v4.x or higher)
Frontend Framework	React.js (v18.x)
IDE	Visual Studio Code
Browser	Google Chrome, Mozilla Firefox, or Edge

SYSTEM DESIGN

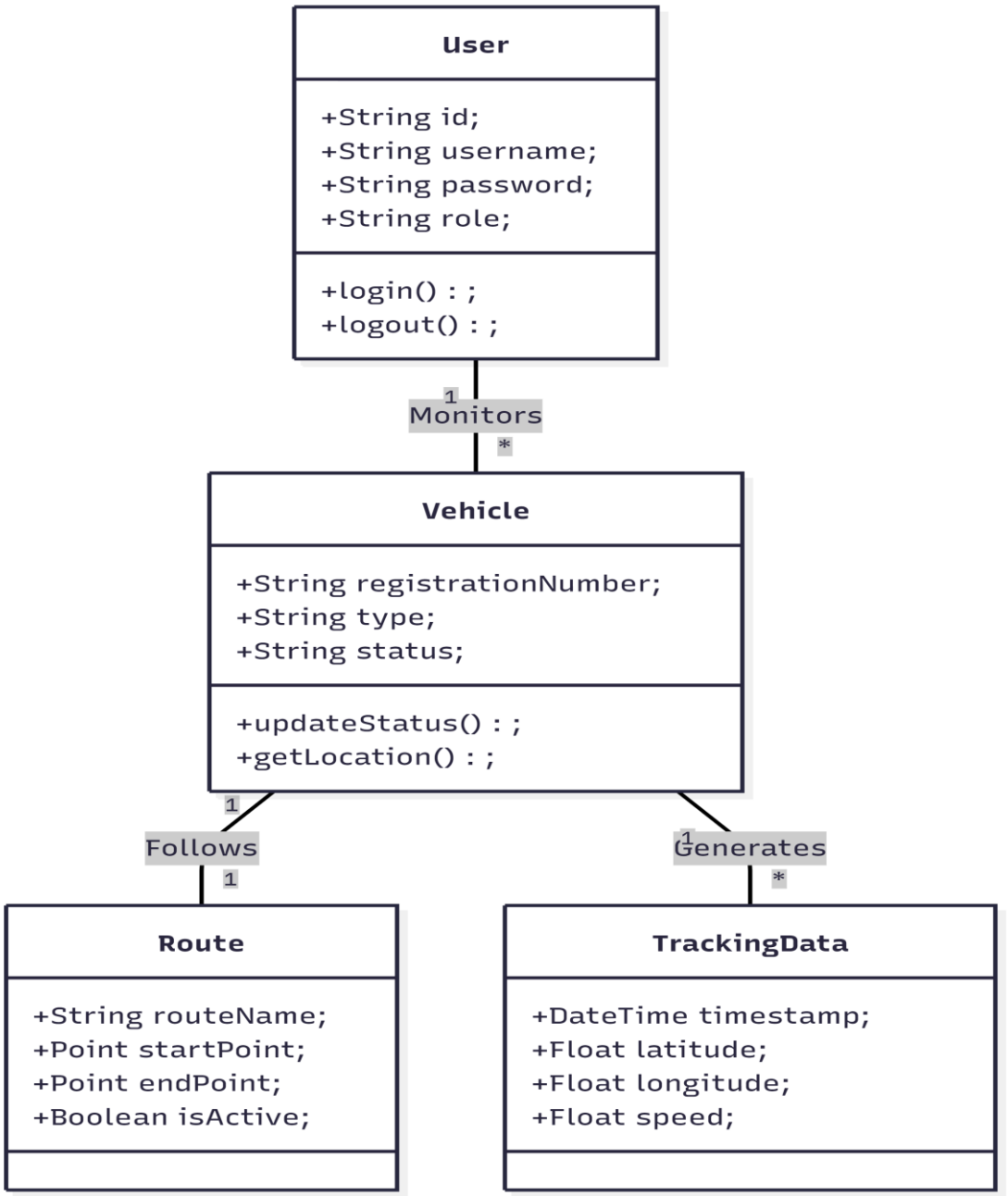
- **DATA FLOW DIAGRAM (DFD) - LEVEL 0**



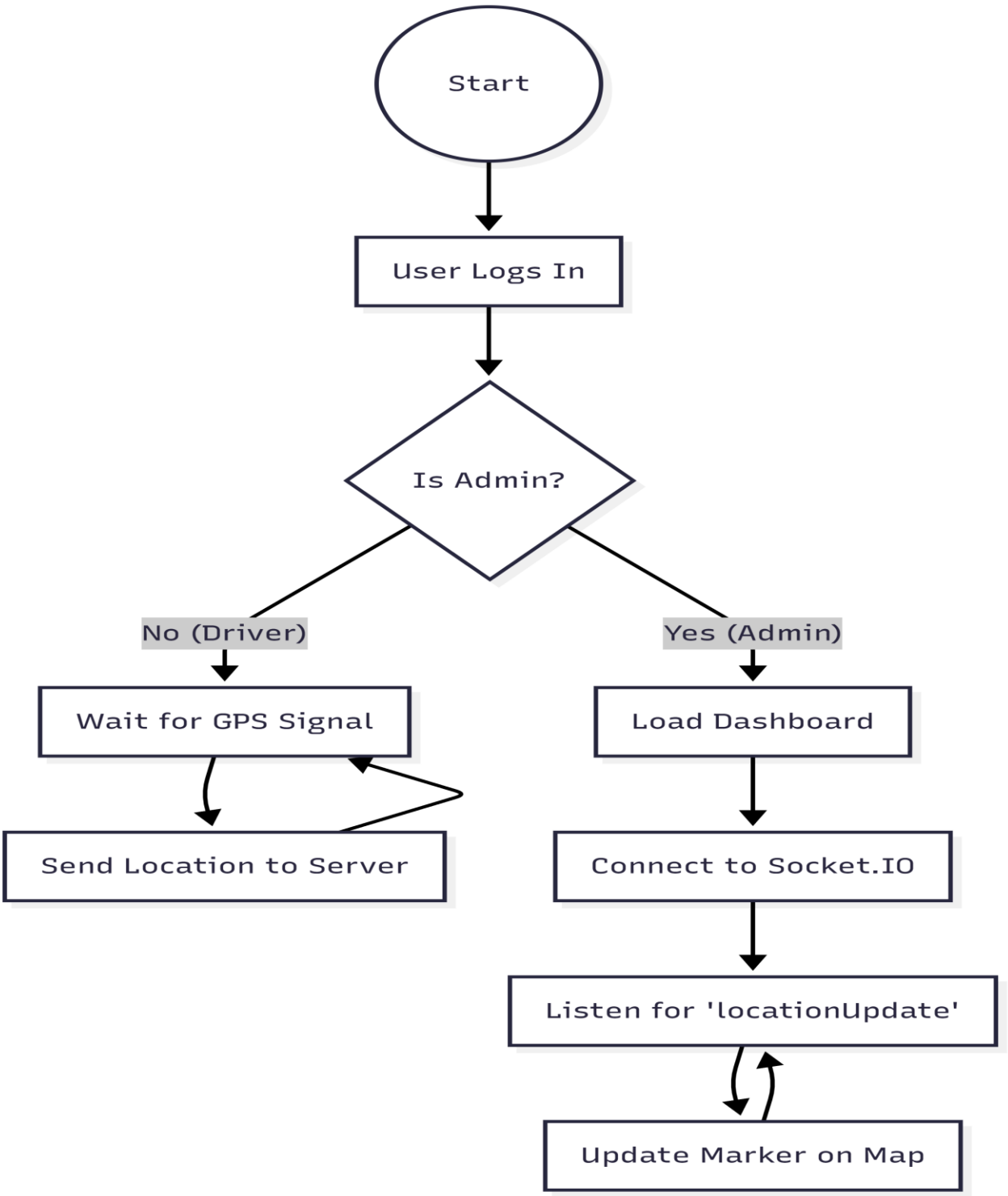
- **USE CASE DIAGRAM**



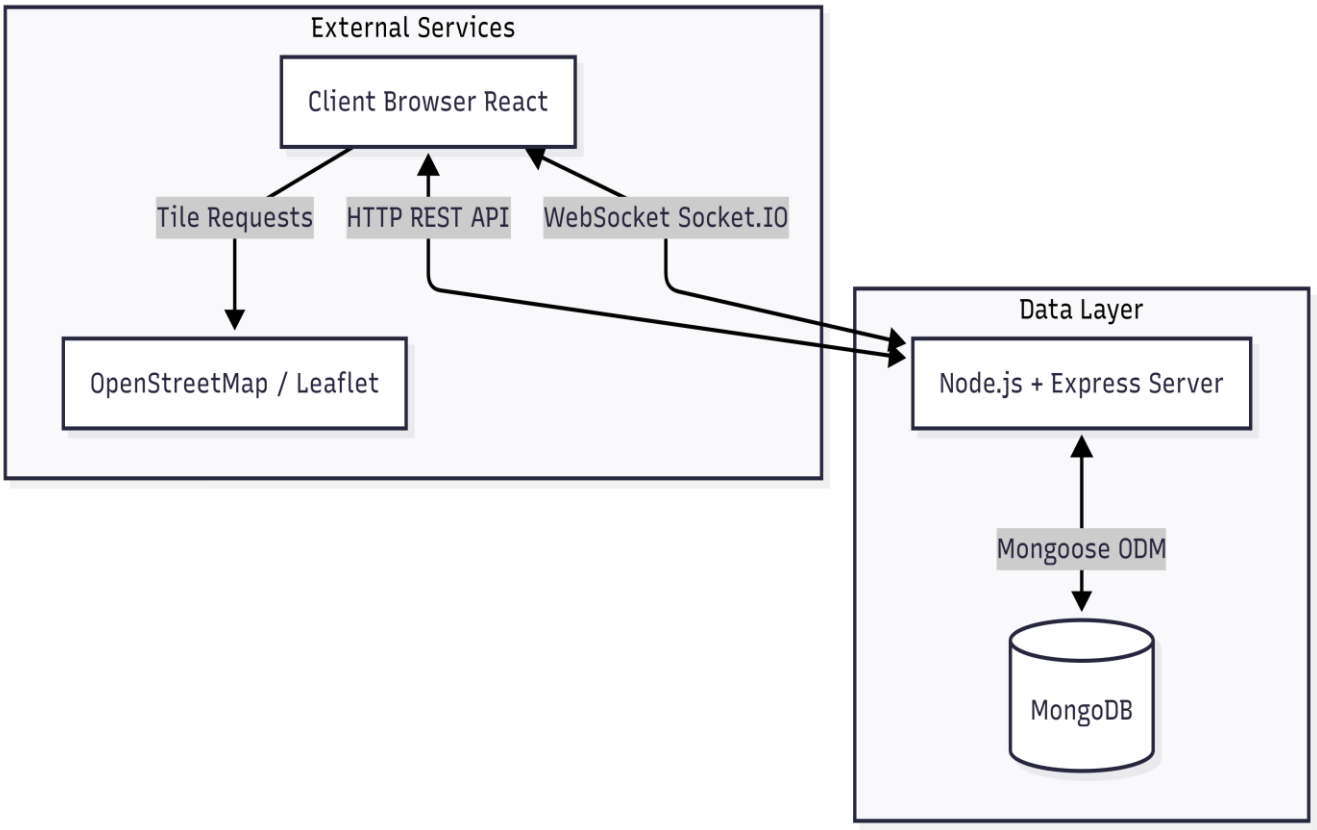
- **CLASS DIAGRAM**



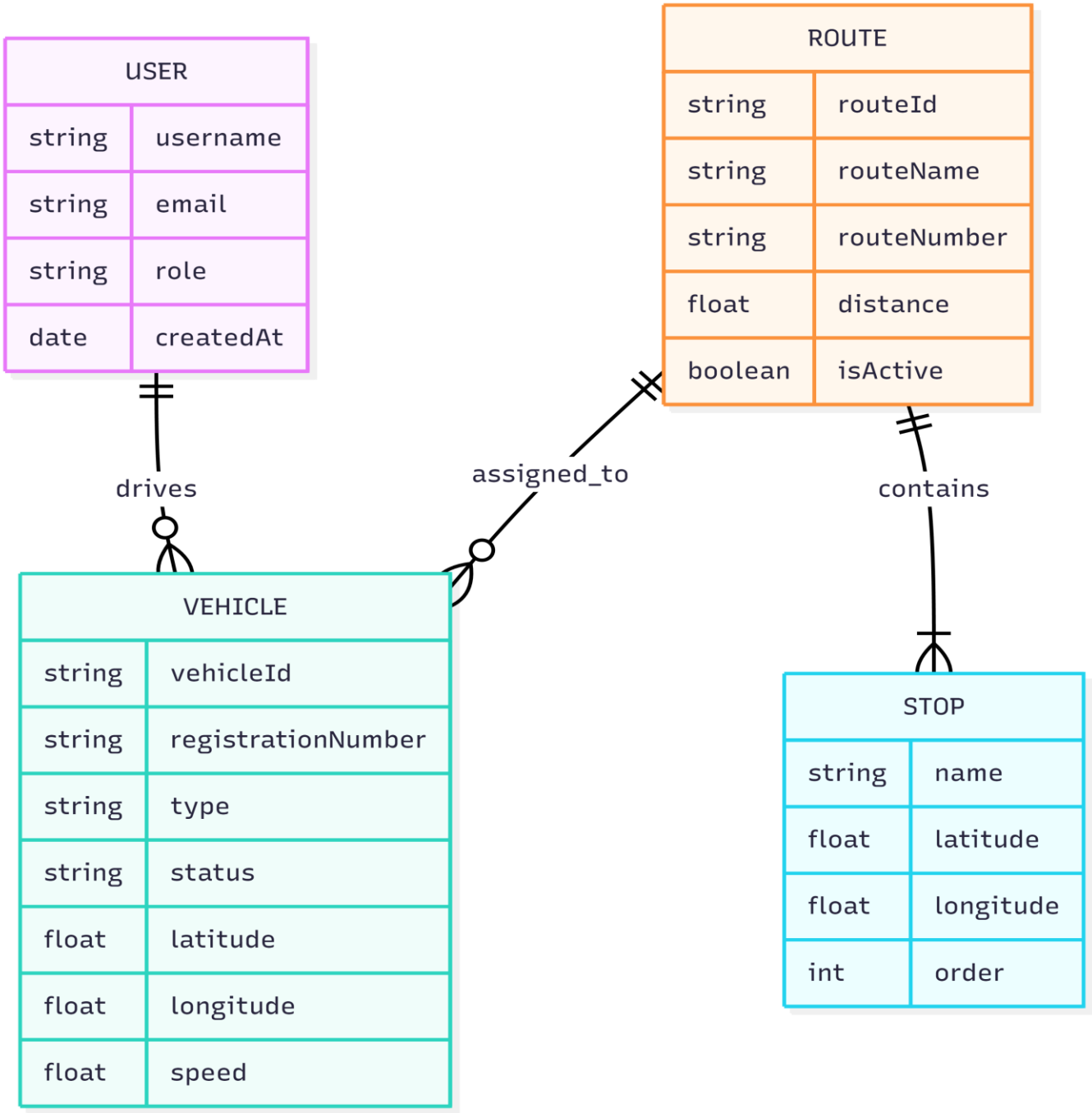
- **ACTIVITY DIAGRAM (TRACKING FLOW)**



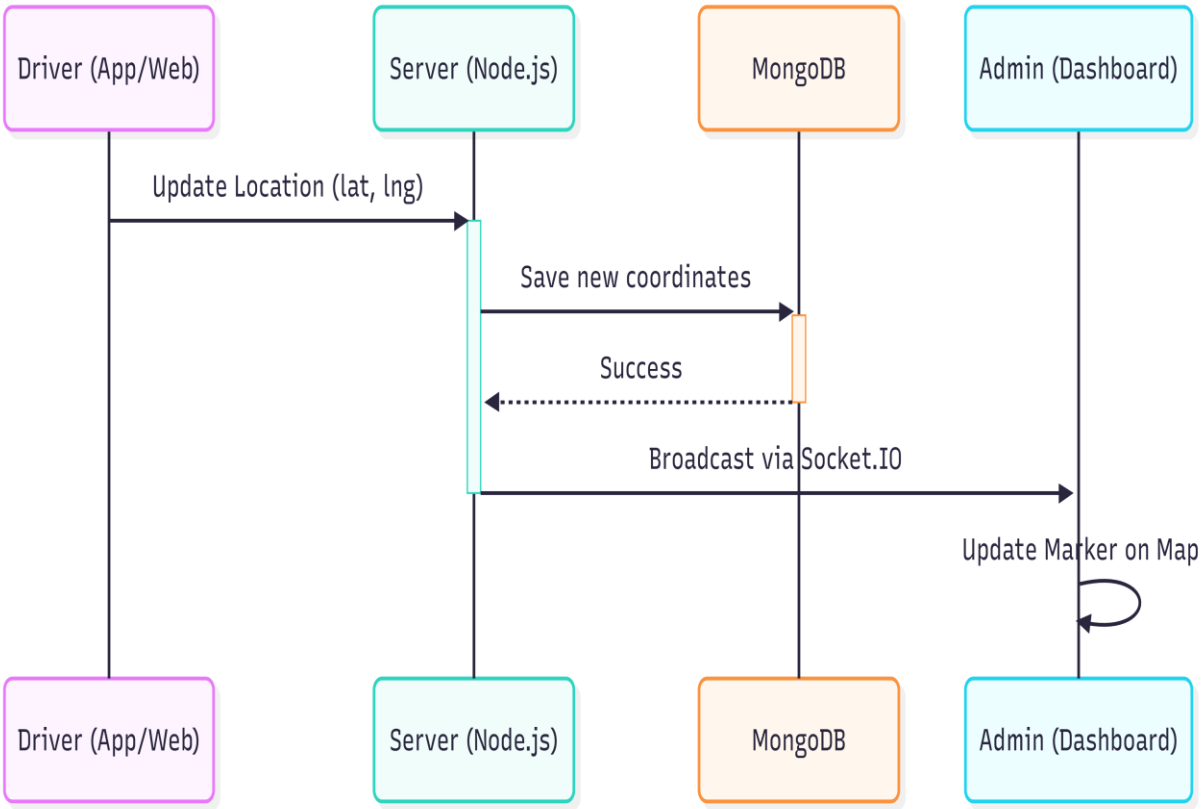
- **SYSTEM ARCHITECTURE DIAGRAM**



- **ER DIAGRAM**


USER
string
username
string
email
string
role
date
createdAt
ROUTE
string
routeId
string
routeName
string
routeNumber
float
distance
boolean
isActive
drives
assigned_to
contains
VEHICLE
string
vehicleId
string
registrationNumber
string
type
string
status
float
latitude
float
longitude
float
speed
STOP
string
name
float
latitude
float
longitude
int
order

- **UML SEQUENCE DIAGRAM**



IMPLEMENTATION

The implementation phase involves translating the system design into executable code. The project is divided into two main directories: marketing (Backend) and frontend (Frontend).

1. BACKEND MODULE

- **PURPOSE:** Serves as the central API and real-time controller.
- **TECHNOLOGIES:** Node.js, Express, Socket.IO.
- **AUTHENTICATION:** Implemented using JSON Web Tokens (JWT). When a user logs in, the `authController` generates a signed token which the client must present for subsequent requests.
- **DATABASE CONNECTION:** The `config/db.js` module utilizes Mongoose to establish an asynchronous connection to the MongoDB instance.
- **REAL-TIME SOCKET:** The `sockets/trackingSocket.js` module initializes the Socket.IO server. It listens for `connection` events, places clients into rooms based on vehicle IDs, and handles `locationUpdate` events to broadcast new coordinates to subscribed admins.

2. FRONTEND MODULE

- **PURPOSE:** Provides the visual interface for interaction.
- **TECHNOLOGIES:** React.js, React Router, Leaflet.
- **STATE MANAGEMENT:** React Hooks (`useState`, `useEffect`) are used to manage local state, such as the list of active vehicles and the current user's profile.
- **MAP INTEGRATION:** The `react-leaflet` library is used to wrap Leaflet maps. Components like `MapContainer`, `TileLayer`, and `Marker` are composed to create the tracking view. Custom icons are used to distinguish between different vehicle types.
- **SOCKET CLIENT:** The frontend establishes a persistent connection to the backend using `socket.io-client`. It listens for live events and updates the component state, triggering a re-render of the map markers.

3. DATABASE SCHEMA IMPLEMENTATION

Mongoose models define the structure of data.

- **USER MODEL:** Stores encrypted passwords and roles.
- **VEHICLE MODEL:** Links to a specific driver and maintains current status.
- **ROUTE MODEL:** Defines geospatial points for planned paths.

- **CODING STANDARDS**
  - **MVC ARCHITECTURE:** The backend follows the Model-View-Controller pattern (implemented as Model-Controller-Route) to separate concerns.
  - **ENVIRONMENT VARIABLES:** Sensitive data like database URIs and JWT secrets are stored in `.env` files, not in the codebase.
  - **ASYNCHRONOUS PROGRAMMING:** `async/await` syntax is used throughout to handle database operations and API calls without blocking the event loop.

TESTING

Testing is a crucial phase to ensure the system operates reliability under expected conditions.

- **TYPES OF TESTING PERFORMED**

1. UNIT TESTING

Individual components were tested in isolation. For the backend, helper functions for data validation were tested. For the frontend, individual React components (like the Navbar or StatusCard) were rendered to ensure they display correct props.

2. INTEGRATION TESTING

This verified the interaction between the API and the Database. We tested whether the `POST /api/vehicles` endpoint correctly created a document in MongoDB and returned the expected JSON response.

3. SYSTEM TESTING

The complete system was tested as a whole. This involved logging in as a driver in one browser window and an admin in another to verify that location updates sent by the driver appeared instantly on the admin's map.

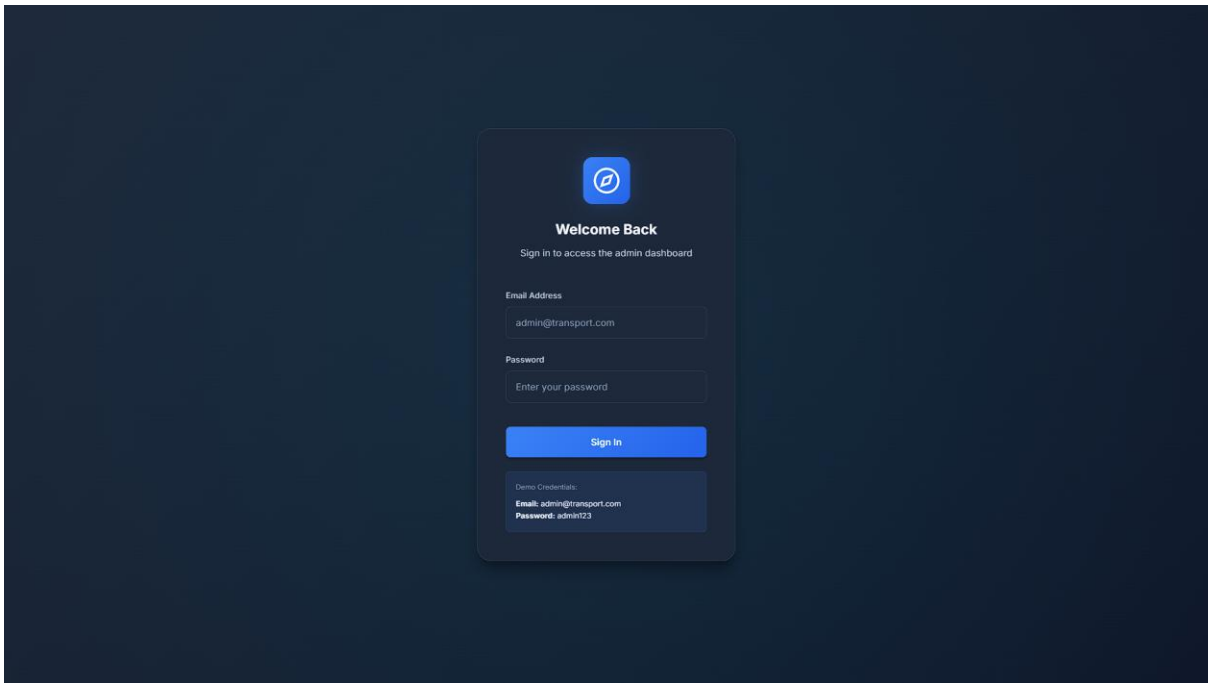
- **TEST CASES**

Case ID	Test Description	Input Data	Expected Result	Actual Result	Status
TC01	User Login (Valid)	Admin Email/Pass	Redirect to Dashboard	User redirected to Dashboard	**Pass**
TC02	User Login (Invalid)	Wrong Password	Error: "Invalid Credentials"	Error displayed correctly	**Pass**
TC03	Create Vehicle	Vehicle Details	Vehicle added to DB	Vehicle visible in list	**Pass**
TC04	Live Tracking	Driver sends coords	Marker moves on Admin map	Marker moved instantly	**Pass**
TC05	Route Assignment	Select Route	Vehicle assigned to Route	Status updated	**Pass**
TC06	API Security	Access w/o Token	401 Unauthorized	Access Denied	**Pass**

OUTPUT SCREENSHOTS

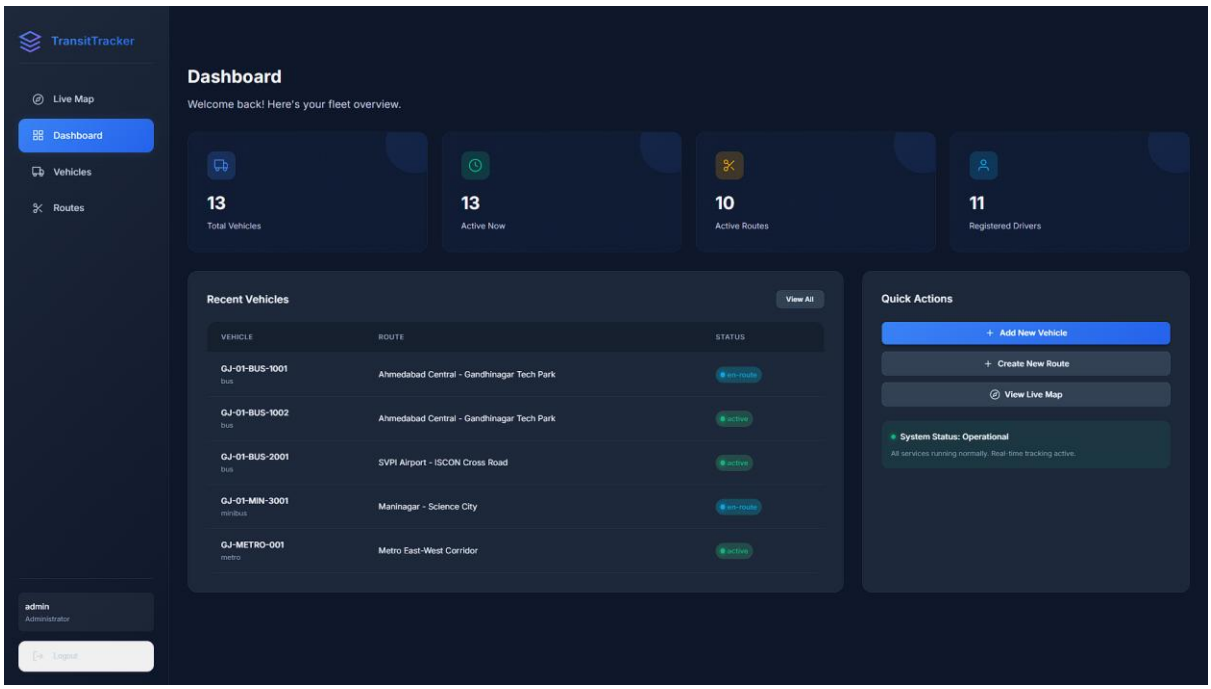
- **ADMIN SIDE :**

1. LOGIN PAGE



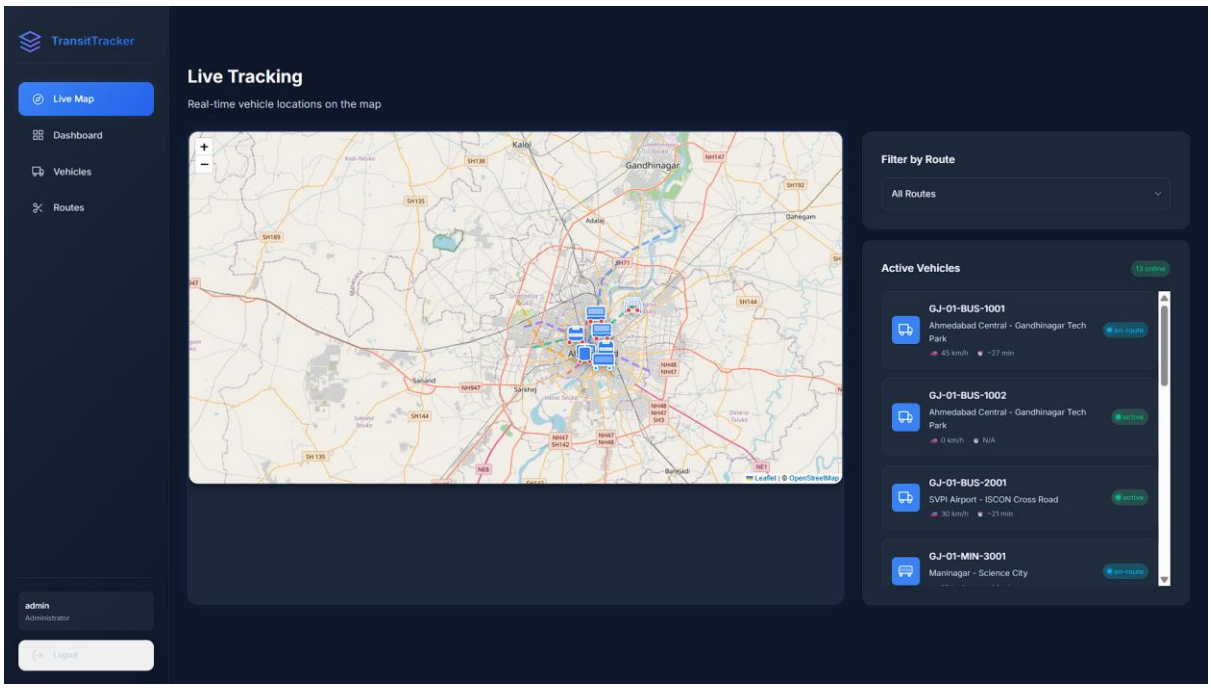
Secure login page for admin access with email and password input, plus demo credentials option.

2. ADMIN DASHBOARD



Overview panel showing total vehicles, active routes, drivers, and recent activity with quick action buttons.

3. LIVE TRACKING MAP



Real-time map showing vehicle locations with route filter and active vehicle list including status and speed.

4. VEHICLE MANAGEMENT

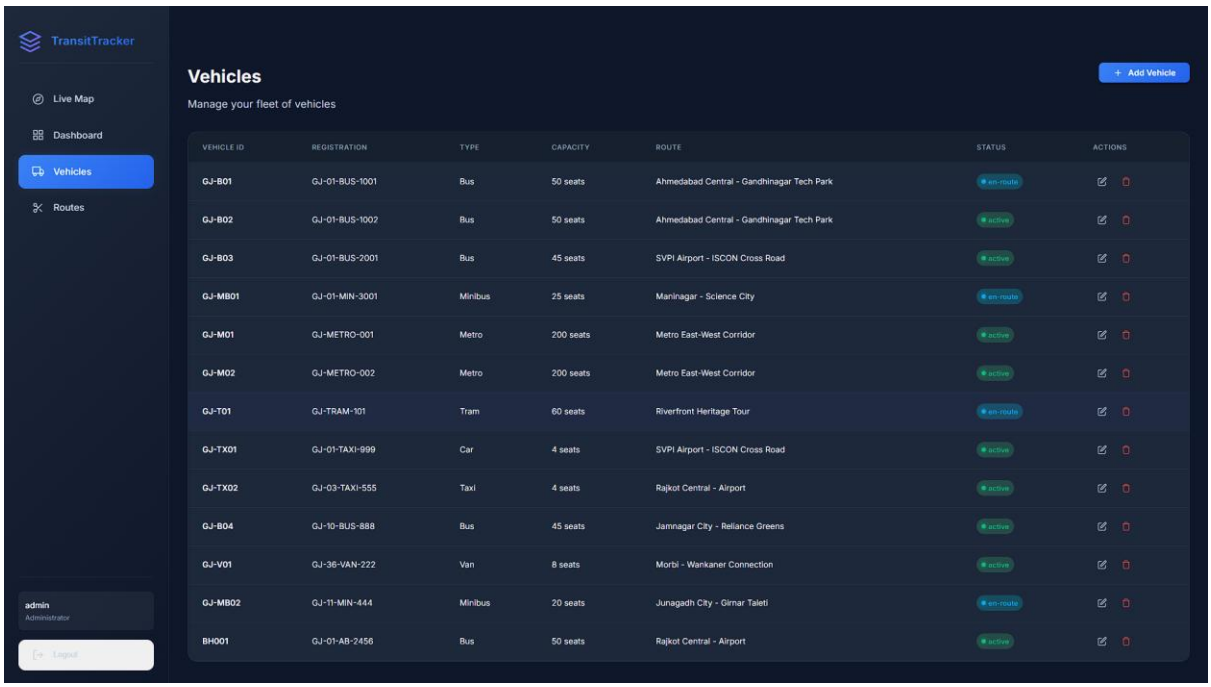
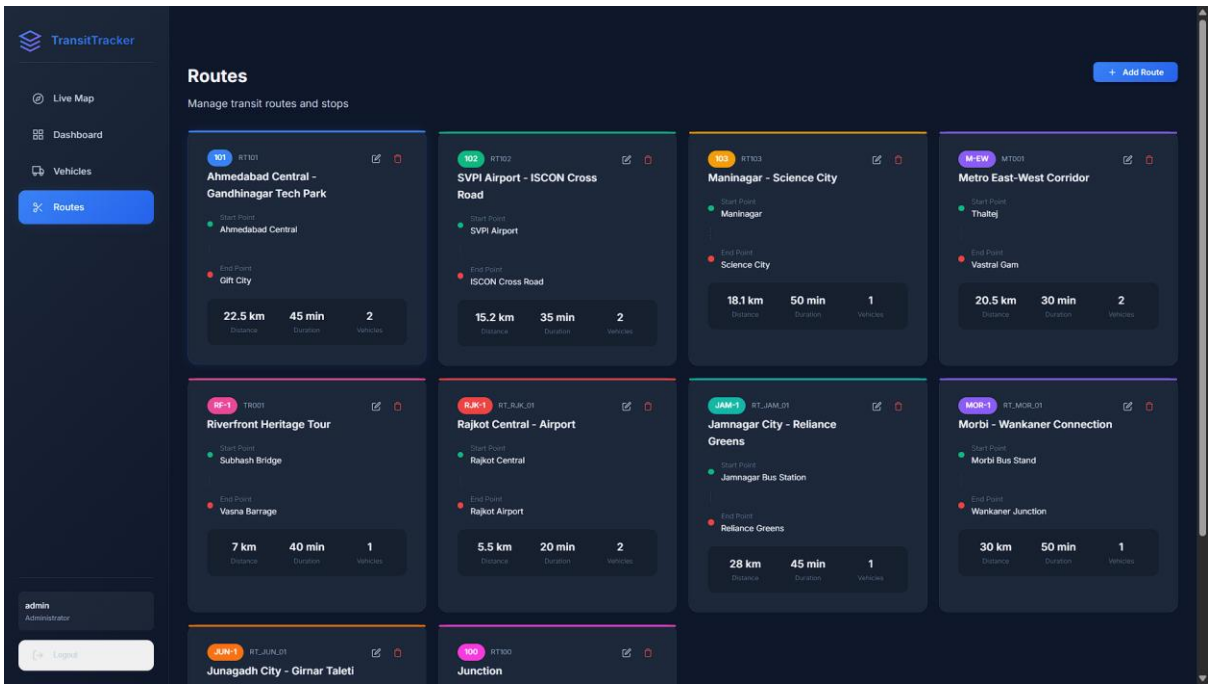


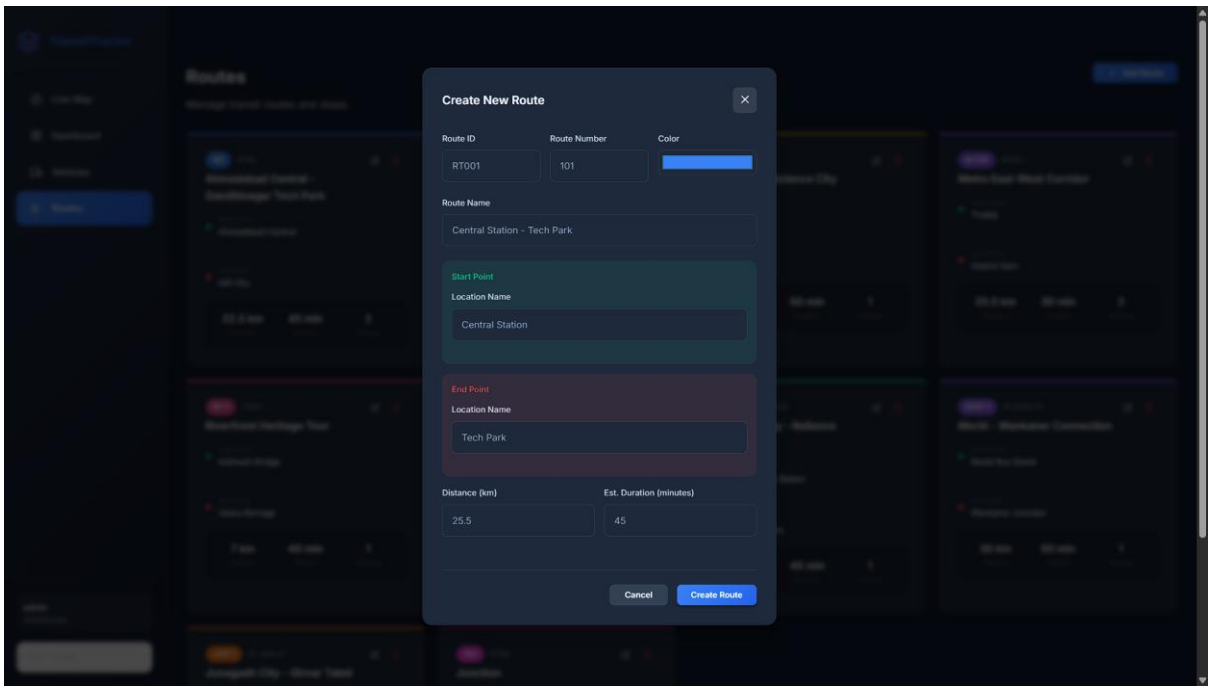
Table view of all vehicles showing registration, type, capacity, assigned route, status, and actions (edit/delete).

5. ROUTES MANAGEMENT



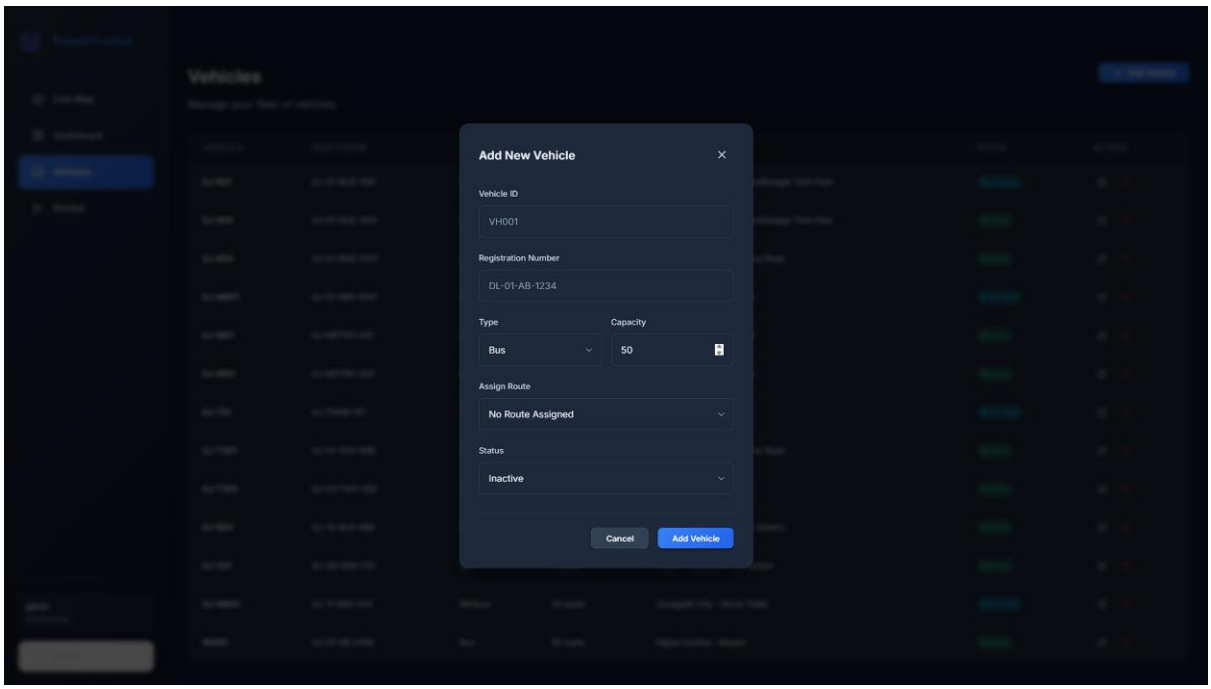
Displays all transit routes in card layout with route details like distance, duration, start/end points, and assigned vehicles.

6. NEW ROUTE CREATE PAGE



Form to create a new route with start/end points, route name, distance, duration, and color coding.

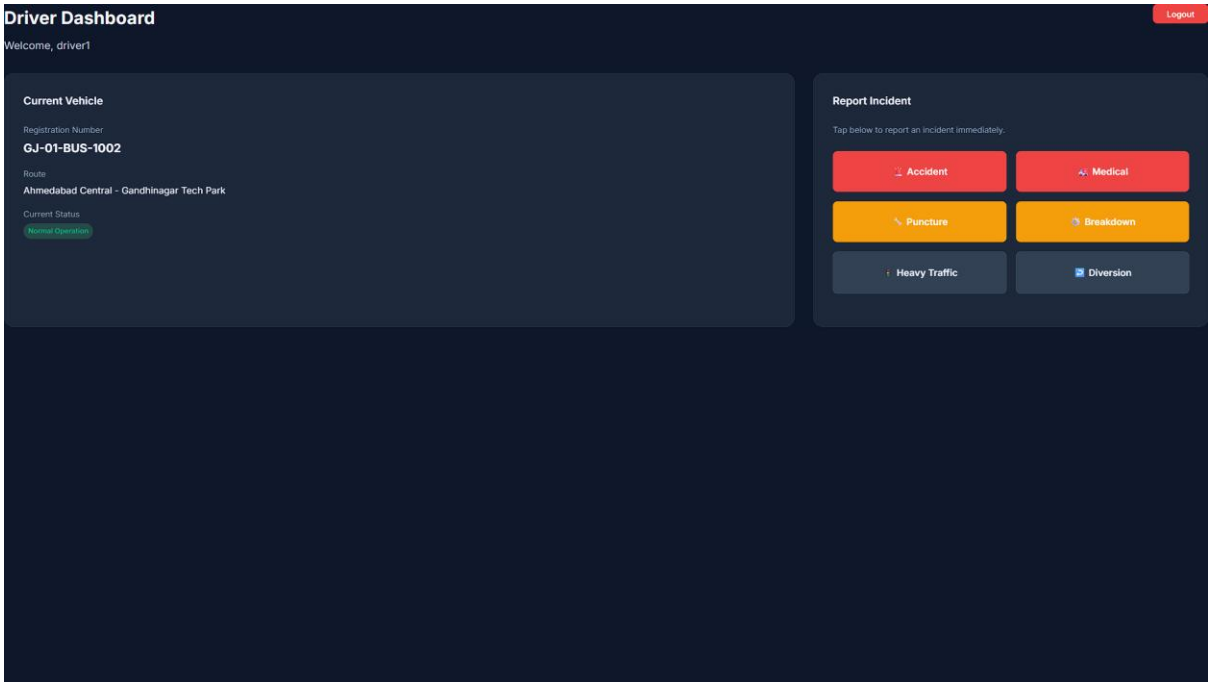
7. NEW VEHICLE REGISTRATION



Form to add a new vehicle including ID, registration number, type, capacity, route assignment, and status.

- **DRIVER SIDE :**

1. DRIVER DASHBOARD



Shows current vehicle details, route info, and status with quick incident reporting buttons like accident, medical, breakdown, etc.

CONCLUSION

The development of the **Vehicle Tracking System** has successfully demonstrated the capability of modern web technologies to solve complex logistical challenges. By integrating the MERN stack with real-time WebSocket communication, the project achieved its primary objective of providing a low-latency, effective monitoring tool for fleet managers.

The system addresses the critical gap in visibility that manual tracking methods suffer from. It offers a secure, reliable, and user-friendly platform that scales well for small to medium-sized fleets. Throughout the development process, valuable insights were gained regarding asynchronous JavaScript, state management in complex React applications, and the handling of geospatial data.

However, the current system allows for tracking based on simulated data or manual web-based updates. Deployment in a real-world scenario would require integration with hardware GPS modules, which presents its own set of challenges regarding connectivity and power management. Despite this limitation, the software infrastructure remains solid and ready for production-grade extensions.

FUTURE SCOPE

The project offers significant scope for future enhancement and expansion:

- **HARDWARE INTEGRATION:** Integrating physical GPS (GSM/GPRS) modules (like Arduino or Raspberry Pi-based trackers) to send data directly to the server APIs, removing the need for drivers to use a web interface.
- **GEOFENCING:** Implementing virtual boundaries. The system could trigger automated alerts (SMS/Email) if a vehicle deviates from its assigned route or leaves a designated area.
- **FUEL MONITORING:** Integrating IoT sensors to monitor fuel levels and detect anomalies that might indicate theft or leakage.
- **MOBILE APPLICATION:** Developing a native mobile app (using React Native) for drivers to improve background location tracking reliability and battery efficiency.
- **ADVANCED ANALYTICS:** Implementing AI/ML algorithms to predict arrival times (ETA) more accurately based on historical traffic patterns and route data.

BIBLIOGRAPHY / REFERENCES

1. *MongoDB, Inc.*"MongoDB Documentation.".

Available: https://docs.mongodb.com/

2. *OpenJS Foundation.* "Node.js Documentation."

Available: https://nodejs.org/en/docs/

3. *Meta Platforms, Inc.* "React - A JavaScript library for building user interfaces.".

Available: https://reactjs.org/

4. *Socket.IO.* "Socket.IO Documentation - Real-time Application Framework.".

Available: https://socket.io/docs/

5. Cheng, F. "Build Real-time Applications with MERN Stack." *IEEE Software*, vol. 35, no. 1, 2021.
6. *Leaflet.* "Leaflet - an open-source JavaScript library for mobile-friendly interactive maps.".

Available: https://leafletjs.com/

7. Freeman, *Pro React 16*. New York, NY, USA: Apress, 2019.

8. Banks, A. and Porcello, E. *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media, 2020.

APPENDIX

A. SAMPLE CONFIGURATION (.ENV)

- **BACKEND CONFIGURATION:**

  PORT=5000

  MONGODB_URI=mongodb://localhost:27017/vehicle-tracking

  JWT_SECRET=project_secret_key_2024

  CLIENT_URL=http://localhost:3000

- **FRONTEND CONFIGURATION:**

  REACT_APP_API_URL=http://localhost:5000

  REACT_APP_SOCKET_URL=http://localhost:5000

B. SAMPLE JSON DATA (VEHICLE OBJECT)


```
{
  "_id": "65b2f...89a",
  "registrationNumber": "KA-01-HH-1234",
  "type": "Truck",
  "status": "active",
  "driver": {
    "name": "John Doe",
    "contact": "+91-9876543210"
  },
  "lastLocation": {
```

"lat": 12.9716,

"lng": 77.5946,

"timestamp": "2024-01-01T10:00:00Z"

}

}